

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent deep learning optimizers such as Schedule-Free optimizers, AdEMAMix, MARS, and Lion differ in motivation, but their empirical behavior is organized by a common quantitative feature: how each method weights the current gradient relative to accumulated momentum. In parallel, theoretical acceleration results for stochastic gradient descent (SGD) in the noise-dominated regime show that acceleration is obtained by decoupling the momentum coefficient from the current-gradient weight. This paper makes that comparison structure explicit. We connect these optimizer families to accelerated SGD and evaluate the resulting predictions on a 150m-parameter language-modeling task. Across the reported comparisons, AdEMAMix is the method whose update most closely matches accelerated SGD, and it attains the strongest performance among the tested optimizers. We then introduce Simplified-AdEMAMix, a single-momentum variant that preserves AdEMAMix-level performance in both the small- and large-batch settings while removing the second momentum term. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recent optimization algorithms for deep learning, including Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024), have been introduced as distinct practical improvements. Viewed through their updates, however, they share a measurable design axis: each changes the relative contribution of momentum and the current gradient in the parameter step. This common axis provides a useful way to compare methods whose empirical performance is otherwise reported under different algorithmic motivations.

A complementary theoretical literature studies when gradient methods accelerate in noisy stochastic settings. Classical heavy-ball and Nesterov momentum accelerate deterministic gradient descent, especially for quadratic objectives, but they do not provide the same acceleration for SGD (Jain et al., 2018; Liu & Belkin, 2020). This negative comparison motivated alternative momentum schemes for noisy optimization (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). The key quantitative pattern across these accelerated SGD variants is that the coefficient on past momentum is decoupled from the weight placed on the current gradient.

Our first contribution is to make the connection between these practical optimizers and accelerated SGD explicit. We show that Schedule-Free SGD is mathematically equivalent to accelerated SGD followed by weight averaging, and we interpret Lion, Schedule-Free AdamW, and AdEMAMix as accelerated-SGD-style updates combined with preconditioning. Prior work has noted related pieces of this relationship (Defazio, 2021), but the comparison among these recent optimizers and the accelerated SGD template has not previously been stated in this form.

We evaluate the comparison on a 150m-parameter decoder-only transformer trained for 15b tokens with a 32k-token batch size, a setting intended to emphasize the noise-dominated regime where accelerated SGD is most relevant. The empirical pattern matches the theory-driven grouping: Schedule-Free AdamW tracks accelerated SGD-based AdamW (Algorithm 3), accelerated variants give modest gains in the small-batch comparison, and the advantage becomes much smaller when the batch size is increased. The large-batch result is important because it separates improvements associated with stochastic acceleration from effects that would persist under substantially reduced gradient noise.

Our main contributions are stated below:

1. We establish precise theoretical connections between accelerated SGD and recent optimizers, including Schedule-Free SGD and AdEMAMix, by identifying the shared momentum/current-gradient weighting structure.
2. We test the comparison on a 150-million-parameter decoder-only transformer and compare AdamW, Schedule-Free AdamW, AdEMAMix, MARS, and related variants under a shared language-modeling setup. In this comparison, AdEMAMix is the closest match to accelerated SGD and achieves the best performance among the evaluated methods.
3. We show that the small-batch advantages associated with accelerated-style updates diminish at large batch sizes relative to Adam, consistent with the predicted dependence on gradient noise. In this regime, Adam with momentum scheduling can match AdEMAMix.
4. At high batch sizes, Schedule-Free AdamW falls substantially behind AdamW with cosine decay; the pattern points to the intrinsic coupling between the momentum and weight-averaging coefficients in Schedule-Free optimizers.
5. We introduce Simplified-AdEMAMix, which matches AdEMAMix across both large- and small-batch regimes while removing the need to maintain two distinct momentum terms.

2 RELATED WORK

We organize the most relevant prior work around the comparison that drives this paper: accelerated SGD methods that change the momentum/current-gradient balance, and practical optimizers whose updates implement closely related balances.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for least-squares problems, establishing a quantitative separation from standard momentum in that setting. Kidambi et al. (2018) simplified the same update rule and formally showed that ordinary momentum does not provide acceleration for this case. Subsequent work (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended the accelerated-SGD comparison to general convex and strongly convex objectives under different theoretical assumptions.

Several practical optimizers can be compared along the same weighting axis. Lucas et al. (2019) used a weighted sum of multiple momentum terms with distinct coefficients in the final update, while Ma & Yarats (2019) proposed an optimizer explicitly motivated by the framework of Jain et al. (2018). More recently, Chen et al. (2023) introduced an optimizer discovered by genetic search that, like accelerated SGD variants, places different weights on the current gradient and the accumulated momentum term. Pagliardini et al. (2024) similarly combines two momentum scales, making it a natural empirical point of comparison for accelerated-SGD-style updates.

3 BACKGROUND

3.1 MOMENTUM

Momentum provides the baseline comparison for all variants studied here. In deterministic settings it accelerates gradient descent, and its update for weights w_t with momentum coefficient β is:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging changes which iterate is evaluated, and therefore it is an important confounder in optimizer comparisons. Rather than returning the final iterate w_T , the method reports a weighted average $\bar{w}_T$ of iterates with averaging weights γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

For consistency across comparisons, every weight-averaging variant in this paper uses coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for a constant $0 \leq \delta \leq 1$, so the effective averaging window is controlled by the same scalar across methods.

3.3 ACCELERATED SGD

We use the following generalized accelerated SGD template to compare methods by their update coefficients:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

Here $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are possibly time-dependent scalar coefficients, and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ marks coefficients in the accelerated-SGD template, making the comparison to practical optimizers explicit.

Setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum, so the template isolates the additional current-gradient weighting used by accelerated variants. Many algorithms from Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) fit this form directly; Appendix B gives the algebraic equivalences used for that comparison.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

This section converts the qualitative resemblance among optimizers into explicit update-level comparisons. We show how Schedule-Free optimizers, Lion, MARS, and AdEMAMix map onto the accelerated-SGD template, then use those coefficient comparisons to motivate a single-momentum variant of AdEMAMix that empirically matches AdEMAMix in both small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a constant-learning-rate optimizer designed to remove explicit learning-rate scheduling. Using the notation of Defazio et al. (2024), its update is:

$$\begin{aligned} y_t &= (1 - \beta) z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1}) x_t + c_{t+1} z_{t+1} \end{aligned}$$

In this notation, y_t is the training point where the gradient is evaluated, while x_t is the evaluation point; this distinction is central to separating optimization dynamics from weight averaging.

We first rewrite the update in terms of y_t and an auxiliary momentum variable m_t , defined as:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Expanding this definition gives the momentum recurrence:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus m_t has exactly the momentum recurrence in Equation (1), with $\beta_{a,t} = 1 - c_t$. The corresponding update for the gradient-evaluation point y_t is:

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Therefore the y_t dynamics match Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$, $\alpha_{a,t} = \gamma(1 - \beta)$, and $w_t = y_t$. Schedule-Free SGD is therefore accelerated SGD at the training point. The remaining question for empirical comparison is the evaluation point x_t , whose dynamics are:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

Thus x_t is a weighted average of the accelerated-SGD iterates y_t . Recursively expanding x_t shows that, for constant c_t , the evaluation weights form an exponential average. Schedule-Free SGD can therefore be compared as accelerated SGD followed by weight averaging, rather than as a wholly separate optimizer family.

This decomposition attributes Schedule-Free SGD's empirical behavior to two separable components:

1. Performance improvements over standard SGD with momentum that come from the accelerated-SGD component.
2. Constant-learning-rate training made viable by weight averaging, specifically tailed weight averaging as described in Section 4.1.3.

Relative to adding acceleration and averaging as separate mechanisms, Schedule-Free SGD/Adam has two practical advantages:

- It avoids extra memory for a separate averaged-weight buffer.
- It removes an explicit weight-averaging coefficient from the hyperparameter sweep.

The same coupling also creates the large-batch failure mode quantified in Section 5.1: the momentum and averaging coefficients cannot be tuned independently when gradient noise is reduced.

4.1.1 CASE: $\beta = 0.0$

When $\beta = 0$, as noted in (Defazio et al., 2024), the accelerated component vanishes and Schedule-Free SGD reduces to standard SGD with weight averaging. Because $c_t = 1/t$, the averaging window begins at the start of training.

4.1.2 CASE: $\beta = 1.0$

When $\beta = 1$, Schedule-Free SGD reduces to standard momentum SGD, again as noted in (Defazio et al., 2024). In this limiting case the momentum coefficient $\beta_{a,t}$ scales as $1 - 1/t$, but the update no longer includes the decoupled current-gradient weight.

4.1.3 CASE: $\beta = 0.9$

For the default Schedule-Free SGD setting $\beta = 0.9$, the coefficients imply the following quantitative structure:

- Since c_t scales as $1/t$, the effective momentum coefficient grows as $1 - 1/t$.
- The current-gradient-to-momentum weight ratio is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- The evaluation iterate averages approximately the most recent 10% of training iterates.

4.2 LION

Lion (Chen et al., 2023) provides another comparison point because it separates the gradient weight used in the update from the momentum coefficient used for state propagation. Its update is:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

In this form, Lion (Chen et al., 2023) can be interpreted as an accelerated-SGD-style update followed by a coordinate-wise sign operation, so the main distinction is the post-processing of the combined direction rather than the underlying momentum/current-gradient split.

4.3 MARS

We next compare MARS-Approx, the practical version of MARS (Yuan et al., 2024), to the accelerated-SGD template. Ignoring bias correction and clipping, its update is:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

Here m_t and v_t are the first- and second-order moment estimates, and x_t denotes the model parameters. To expose the comparison with Equation (1), rewrite the update using $\hat{m}_t = m_t - \gamma g_t$:

$$\begin{aligned}
c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\
\hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma)g_t \\
v_t &= \beta_2 v_{t-1} + (1 - \beta_2)c_t^2 \\
x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon}
\end{aligned}$$

This rewrite shows that the first-moment update has the accelerated-SGD structure, while MARS differs from AdamW in its preconditioning term. The experiments in Section 5 therefore compare both the acceleration-style direction and the different preconditioner used by MARS.

4.4 AdEMAMix

AdEMAMix (Pagliardini et al., 2024) is the closest practical match to accelerated SGD-based AdamW among the optimizers we study. The key difference is representational: accelerated SGD combines the current gradient with one momentum term, whereas AdEMAMix maintains two momentum terms with different coefficients and combines them in the final update. Algorithm 1 states the algorithm.

For a cleaner coefficient comparison, we consider the $\beta_1 = 0$ variant of AdEMAMix. Pagliardini et al. (2024) reported that this variant is nearly performance-equivalent to the full method at small batch sizes, and our experiments in Section 5 show the same pattern. With $\beta_1 = 0$, AdEMAMix aligns directly with Equation (1). Moreover, the AdEMAMix schedules for β_3 , the slow-momentum coefficient, and α , the relative weight on the current-gradient component, closely mirror the schedules proposed for accelerated SGD (Gupta et al., 2023).

In smooth convex optimization, stochastic acceleration requires the momentum coefficient to scale as:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximately matches this requirement by increasing β_3 over training.

The normalization differs across the two formulations. In accelerated SGD, momentum is accumulated in the standard form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas Algorithm 1 uses an EMA-style slow momentum:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in accelerated SGD grows proportionally to t . AdEMAMix compensates for its EMA normalization by scaling α proportionally to t . This coefficient-level match explains why AdEMAMix is the strongest empirical method in the small-batch comparison shown in Figure 1.

At large batch sizes, the same coefficient comparison predicts a limitation. AdEMAMix with $\beta_1 = 0.0$ loses performance, as also reported in Pagliardini et al. (2024), because Gupta et al. (2023) suggests that the current-gradient weight should decrease as batch size grows. AdEMAMix instead keeps a fixed weight of 1 on the current gradient, which likely accounts for the large-batch degradation.

The next subsection uses this observation to define a simpler variant: it keeps an explicit current-gradient weight, removes the schedule on α , and maintains only one momentum term. The empirical comparisons then test whether this reduced update preserves AdEMAMix performance across both batch-size regimes.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

-
- 1: Sample batch B_t .
 - 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
 - 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
 - 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v}} + \epsilon}$
 - 5: $w \leftarrow w - \eta N$
 - 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
 - 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
 - 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$
-

4.5 SIMPLIFIED ADEMAMIX

Motivated by the coefficient comparison above, Simplified-AdEMAMix removes the second momentum buffer and the schedule on α while preserving the key accelerated-SGD-style split between momentum and the current gradient. Algorithm 2 presents the update using theory-style momentum rather than EMA-style momentum. The final direction assigns a fixed weight α to the current gradient; setting $\alpha = 0$ recovers standard Adam after the corresponding transformations of η and β_1 . The experiments in Section 5 show that this reduced update matches AdEMAMix in both the small- and large-batch comparisons.

5 EXPERIMENTS

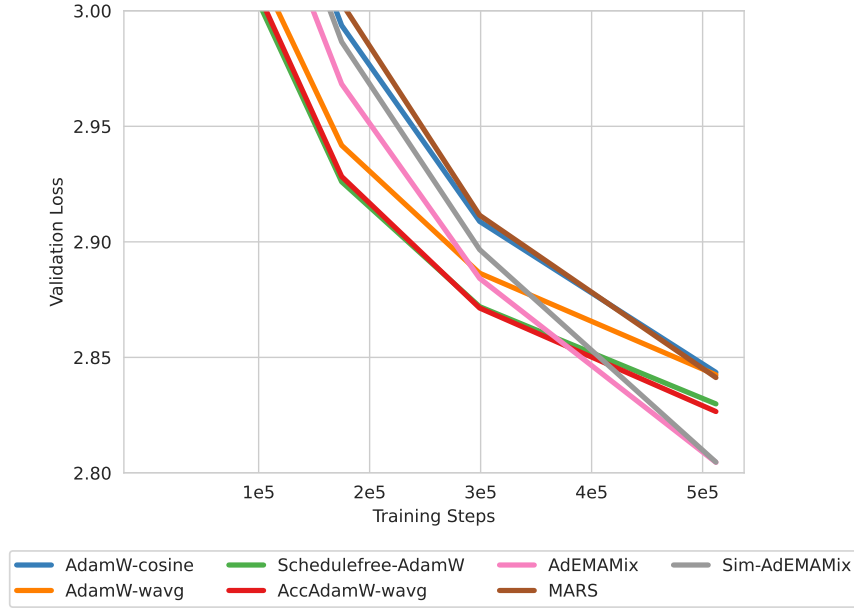


Figure 1: Best validation runs for the optimizer comparison in Section 5 on the 150m-parameter decoder-only language model. AdEMAMix and Simplified-AdEMAMix are the strongest curves in this small-batch setting, consistent with their closest coefficient-level match to accelerated SGD variants.

We evaluate the theoretical grouping on a 150-million-parameter decoder-only transformer trained for language modeling on C4. The small-batch setting uses sequence length 1024, batch size 32, and 15 billion training tokens ($\approx 5 \times$ Chinchilla), giving a noise-dominated regime where accelerated-SGD-style differences should be visible.

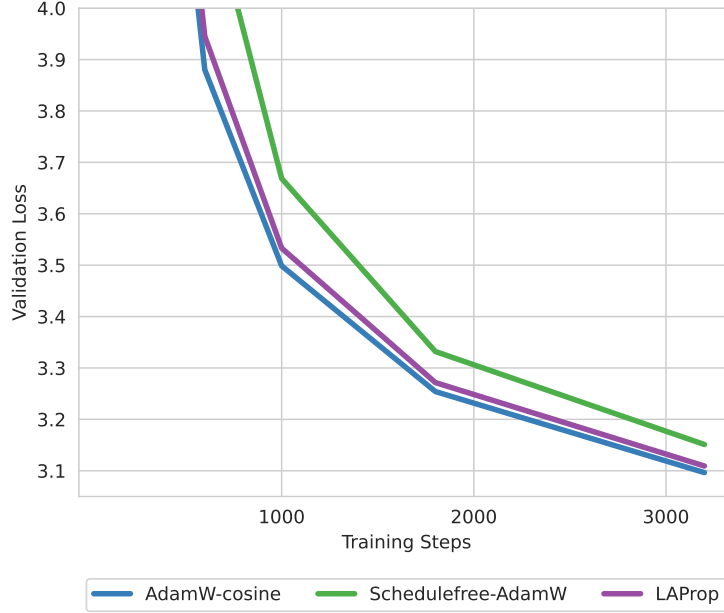


Figure 2: Best-run comparison at higher batch size for AdamW with cosine decay, Schedule-Free AdamW, and LAMProp. The comparison isolates whether the Schedule-Free gap comes from its preconditioning order or from the coupled momentum/averaging coefficients discussed in Section 5.1.

The comparison includes the following optimizers and variants, chosen to separate standard AdamW, weight averaging, Schedule-Free updates, accelerated-SGD-style AdamW, and two-momentum methods:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

Section A reports the hyperparameter sweeps used for each method, so the plotted comparisons are based on the best runs found within the stated search grids.

Figure 1 shows a consistent ordering aligned with the update-level analysis. Schedule-Free AdamW and accelerated AdamW with tailed weight averaging perform similarly, supporting the claimed equivalence between Schedule-Free dynamics and accelerated SGD plus averaging. Both methods improve over AdamW with cosine decay and AdamW with tailed weight averaging, while AdEMAMix and Simplified-AdEMAMix achieve the best validation performance among the tested methods. This margin pattern is consistent with AdEMAMix-style updates being the closest empirical instantiation of the accelerated-SGD coefficient structure.

5.1 LARGE BATCH SIZE EXPERIMENTS

The small-batch experiments emphasize noisy gradients; the large-batch experiments test whether the same optimizer ordering persists when gradient noise is reduced. In this setting, models are trained with a batch size of 1 million tokens for 3 billion tokens ($\approx$ Chinchilla scale), allowing a direct comparison of practical significance across batch-size regimes.

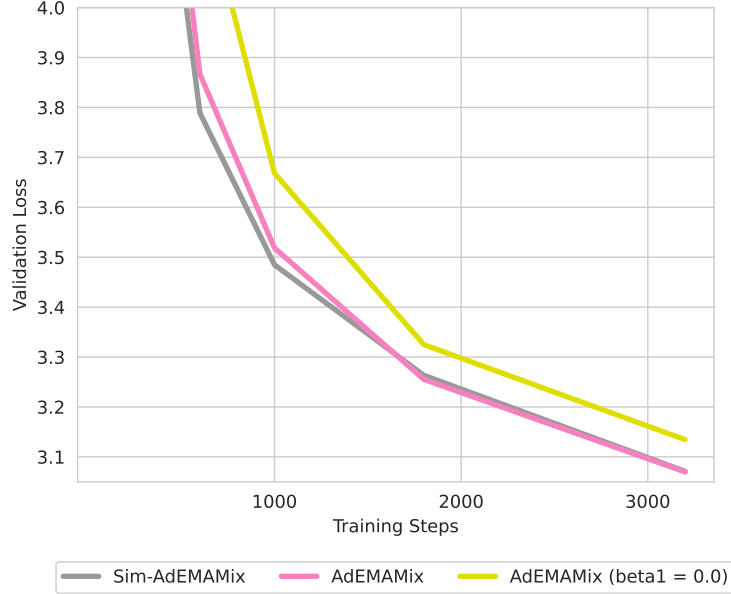


Figure 3: Best-run comparison at higher batch size for AdEMAMix with and without $\beta_1 = 0.0$ and for Simplified-AdEMAMix. The curves test whether reducing the current-gradient emphasis preserves AdEMAMix-level performance in the large-batch regime described in Section 5.1.

Schedule-Free AdamW: Figure 2 shows that Schedule-Free AdamW is substantially worse than AdamW in the high-batch comparison. The gap is consistent with the coefficient analysis: Schedule-Free AdamW ties the weight-averaging and momentum coefficients together, but at high batch sizes the best momentum value is much smaller than $1 - 1/t$. A scaling factor of the form $\approx 1 - r/t$ with $r \geq 1$ can reduce the momentum, but increasing r also shortens the effective weight-averaging window, so the two quantities cannot be optimized independently.

The second possible explanation is the order of momentum and preconditioning. AdamW applies momentum before preconditioning, while Schedule-Free AdamW applies preconditioning before momentum and is therefore closer to LARProp (Ziyin et al., 2021). Figure 2 shows that AdamW and LARProp are comparable, so this ordering difference does not explain the main performance margin. The evidence instead points to the coupled momentum/averaging coefficients.

AdEMAMix: In the large-batch setting, Figure 3 reproduces the pattern observed by Pagliardini et al. (2024): setting $\beta_1 = 0.0$ causes a clear performance drop relative to using two momentum terms. This result aligns with the accelerated-SGD comparison because AdEMAMix assigns a fixed weight of 1 to the current gradient, while theoretical accelerated SGD variants (Gupta et al., 2023) require the current-gradient weight to shrink as batch size increases.

Figure 3 also shows that Simplified-AdEMAMix matches AdEMAMix in this higher-batch comparison while using only one momentum term. The best matched performance occurs at $\alpha = 0.0$, so in this regime Simplified-AdEMAMix reduces to standard Adam with momentum scheduling. This result sharpens the practical conclusion: the second momentum buffer is not needed to match AdEMAMix under the tested large-batch condition.

6 CONCLUSION

This work connects recent optimizers to accelerated SGD by comparing the quantitative weights placed on momentum and the current gradient. Schedule-Free optimizers, AdEMAMix, MARS, and Lion all fit naturally into this comparison, and the empirical results follow the predicted structure: in the small-batch language-modeling setting, AdEMAMix is the closest match to accelerated SGD and achieves the strongest performance among the evaluated methods.

The large-batch experiments further clarify which gains are robust across evaluation conditions. Schedule-Free AdamW falls behind AdamW when its coupled averaging and momentum coefficients become poorly matched to the high-batch regime, while Simplified-AdEMAMix matches AdEMAMix across the tested small- and large-batch settings. Thus the main practical outcome is a simpler optimizer with AdEMAMix-level performance in these comparisons and without the extra memory overhead of a second momentum buffer.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzba->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

The small-batch comparisons use the following hyperparameter grids. These grids define the search space for the best-run curves reported in Figure 1 and make the optimizer, metric condition, and batch-size regime explicit.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999 respectively matching the default values. We note that for larger batch sizes it is common to use $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This performed worse than warmup of 51.2k steps.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The large-batch comparisons use the following sweeps, corresponding to the best-run curves in Figure 2 and Figure 3:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LAMProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

Equation (1) gives the accelerated SGD template used throughout the main text. This appendix verifies that the methods of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) fall within that same form, supporting the coefficient-level comparisons used in the empirical discussion.

B.1 AGNES

For AGNES (Gupta et al., 2023), the update is:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n and the final function value is evaluated at x_n . Rewriting the equations gives

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} follows an update of the form in Equation (1), making AGNES part of the same comparison class.

B.2 ASGD

For the ASGD method of Jain et al. (2018), the update is:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} and performance is evaluated on the tail-averaged x iterate.

The same update can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

These equations match the form of Equation (1), so ASGD is included in the accelerated-SGD template used for comparison.

B.3 MASS

For MaSS (Liu & Belkin, 2020), the update is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t and the function value is evaluated at w_t . Rewriting yields:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The rewritten equations again follow Equation (1), so MaSS shares the same coefficient structure.

B.4 SGD WITH NESTEROV ACCELERATION

For the SGD method with Nesterov acceleration from Vaswani et al. (2019), the update is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations can be rewritten in the comparison form as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$